

ISTANBUL TECHNICAL UNIVERSITY
COMPUTER ENGINEERING DEPARTMENT

BLG 222E
COMPUTER ORGANIZATION
PROJECT 3 REPORT

CRN : 23728 & 23727

LECTURER : Prof. Dr. Mustafa Ersel Kamaşak &
Prof. Dr. Gökhan İnce

GROUP MEMBERS:

150230007 : Celal Gündoğdu

150230008 : İsmail Efe Çelik

SPRING 2026

Contents

1	INTRODUCTION	1
2	CONTROL UNIT DESIGN	1
2.1	Overview	1
2.2	Instruction Fetch: Two-Cycle IR Loading	1
2.3	Synchronous Active-Low Reset	2
2.4	Instruction Format and Decoding	3
2.4.1	Address-Referenced Format	3
2.4.2	Register-Referenced (Non-Address) Format	3
2.5	Instruction Processing Across Clock Cycles	4
2.5.1	Branch Instructions (Opcodes 0x00–0x06)	4
2.5.2	Single-Operand ALU Instructions (Opcodes 0x07–0x0E)	5
2.5.3	Two-Operand ALU Instructions (Opcodes 0x0F–0x15)	6
2.5.4	Data Movement and Memory Instructions	6
2.6	CPU System Schematic	8
2.7	Summary of Control Signal Generation	10
3	INSTRUCTION AND CLOCK CYCLE ANALYSIS	11
3.1	Complete Instruction Set with Cycle Counts	11
3.2	Reasoning Behind Cycle Counts	12
4	SIMPLE PROGRAM ANALYSIS	14
4.1	Program Description	14
4.2	Total Clock Cycle Calculation	15
4.2.1	Initialization Phase (before loop)	15
4.2.2	Loop Body	15
4.2.3	Finalization	16
4.2.4	Grand Total	16
5	SIMULATION RESULTS	17
5.1	Per-Instruction Simulation	17
5.2	Factorial Program Simulation	18
6	DISCUSSION	20
7	CONCLUSION	20
8	TASK DISTRIBUTION	21

1 INTRODUCTION

In this report, we will go through the process of developing and verification of a 16 bit CPU system used in the BLG 222E Computer Organization class. The CPU was implemented incrementally in three different projects. While Project 1 introduced the registers and arithmetic logic unit, Project 2 brought together all these modules in a single data-path, and Project 3 completed the CPU by adding a control unit for fetching, decoding, and executing instructions. Our CPU consists of only two phases, one for fetching the instruction and another for executing it. Instructions have a width of 16 bits, however, since our instruction memory has a width of 8 bits, we need two read cycles to form an instruction in the Instruction Register. After receiving the instruction, the control unit selects the appropriate data path signals according to the instruction opcode and operand structure. All components have been implemented using Verilog, and then simulated via Icarus Verilog. The rest of this report is organized as follows: Section 2 will focus on the architecture of our control unit and signal generation logic.

2 CONTROL UNIT DESIGN

2.1 Overview

Control Unit uses a *hardwired* design. All control signals are generated by combinational circuits whose inputs consist of the timing state, the instruction, and ALU flag bits. Microprogram ROM is not used in the control unit design. Timing state registers are designed using a one-hot 12-bit shift register called **T**. During every rising clock edge, the active bit in the register shifts one place to the left. This causes the incrementing of timing states T_0, T_1, T_2 , etc., till the assertion of **T_Reset** resets them back to T_0 .

The **set_defaults** task is responsible for setting each control signal to its default (inactive) value before the evaluation of each combinatorial phase starts. This ensures that only the signals whose values are set within a specific timing point will be activated, avoiding any unintended write operations.

2.2 Instruction Fetch: Two-Cycle IR Loading

Because the instruction memory is 8 bits wide while each instruction is 16 bits, the fetch stage spans two clock cycles arranged in a little-endian fashion:

1. T_0 — **Fetch LSB**. The Program Counter (PC) is placed on the address bus through **ARF_OutDSel**. The instruction memory chip select is activated (**IMU_CS**= 0) with **IMU_LH**= 0, directing the byte read from ROM into the *lower* half of the Instruction

Register (IR[7:0]). Simultaneously, the PC is incremented via the ALU (pass-through increment, ALU_FunSel= 0x10) and written back through MuxB.

2. T_1 — **Fetch MSB**. The same sequence repeats with IMU_LH= 1, so the second byte is loaded into IR[15:8]. The PC is incremented a second time. After this clock edge the full 16-bit instruction word is available in the IR and the PC points to the next instruction.
3. T_2 onward — **Execute**. The opcode field IR[15:10] is decoded and the appropriate control signals are generated. Simple instructions complete in a single execute cycle (T_2); more complex ones (e.g., memory load/store, stack operations) extend through T_3 – T_6 before T_Reset is asserted.

Table 1 summarises the key signal values during the fetch phase.

Table 1: Control signal settings during the two-cycle instruction fetch.

Signal	T_0 (Fetch LSB)	T_1 (Fetch MSB)
ARF_OutDSel	2'b00 (PC)	2'b00 (PC)
ARF_OutCSel	2'b00 (PC)	2'b00 (PC)
IMU_CS	0 (active)	0 (active)
IMU_LH	0 (low byte)	1 (high byte)
IR_Enable	0 (active)	0 (active)
MuxCSel	1 (ARF_OutC)	1 (ARF_OutC)
ALU_FunSel	0x10 (A+1)	0x10 (A+1)
MuxBSel	2'b00 (ALUOut)	2'b00 (ALUOut)
ARF_RegSel	3'b110 (PC)	3'b110 (PC)
ARF_FunSel	0 (Load)	0 (Load)

2.3 Synchronous Active-Low Reset

The signal **Reset** is low-active and works in a synchronous fashion with the clock. If **Reset**=0, then the combinational part avoids regular decoding and sets all the enable signals for the registers to the active level while setting all the **FunSel** inputs to:

- RF_RegSel= 4'b0000 and RF_ScrSel= 4'b0000 — selects all four general-purpose registers and all four scratch registers (active-low enables).
- RF_FunSel= 1 — the “clear” function, which loads zero into each selected register on the next rising edge.

- `ARF_RegSel` = 3'b000 — selects PC, AR, and SP simultaneously.
- `ARF_FunSel` = 1 — clears all address registers.
- `T_Reset` = 1 — forces the timing counter back to T_0 so that the first instruction fetch begins as soon as reset is de-asserted.

Because the clear takes effect on the next positive clock edge, all registers transition to zero in a single cycle. After reset is released the machine enters the normal T_0 fetch state with `PC` = 0x0000, beginning execution from the first address in instruction memory.

2.4 Instruction Format and Decoding

The 16-bit instruction word uses two distinct encoding formats depending on whether the instruction references a memory address or operates purely on registers.

2.4.1 Address-Referenced Format

Table 2: Address-referenced instruction encoding (branches, LDA, STA, LDT, STT, IMM).

Bits [15:10]	Bits [9:8]	Bits [7:0]
Opcode (6 bits)	RSEL (2 bits)	ADDRESS / VALUE (8 bits)

For branch instructions (0x00–0x06), the 8-bit address field is zero-extended to 16 bits and loaded directly into PC when the branch condition is satisfied. The RSEL field is unused for branches. For LDA (0x1E), STA (0x1F), LDT (0x20), and STT (0x21), the address or offset occupies the lower 8 bits while RSEL picks one of the four general-purpose registers (R1–R4). The IMM instruction (0x17) similarly uses the ADDRESS field as an 8-bit immediate value and RSEL to choose the destination register.

2.4.2 Register-Referenced (Non-Address) Format

Table 3: Register-referenced instruction encoding (ALU operations, MOV, LDR, STR).

Bits [15:10]	Bits [9:7]	Bits [6:4]	Bits [3:1]	Bit [0]
Opcode (6 bits)	DSTREG (3 bits)	SREG1 (3 bits)	SREG2 (3 bits)	unused

Each 3-bit register specifier uses its MSB to select between the Register File (bit = 1) and the Address Register File (bit = 0). The lower two bits then index a specific

register within the chosen file. For single-operand ALU operations (0x07–0x0E), only DSTREG and SREG1 are meaningful. Two-operand operations (0x0F–0x15) additionally use SREG2. The MOV instruction (0x16) copies SREG1 into DSTREG. For LDR (0x1C) and STR (0x1D), DSTREG/SREG1 point to the data register while the Address Register (AR) supplies the memory address.

2.5 Instruction Processing Across Clock Cycles

In order to illustrate the cycle-by-cycle functioning of the control unit, the control signals for different types of instructions are listed below. Each line represents one clock cycle; “—” denotes that the signal is maintained at its default value.

2.5.1 Branch Instructions (Opcodes 0x00–0x06)

Branch instructions complete in a single execute cycle. The condition is evaluated combinationally from the stored ALU flags, and if the branch is taken the 8-bit address field is loaded into the PC.

Table 4: Cycle-by-cycle signal trace for branch instructions.

Cycle	Actions and Key Signals
T_0	Fetch LSB: IMU_CS=0, IMU_LH=0, IR_Enable=0; PC incremented via ALU.
T_1	Fetch MSB: IMU_CS=0, IMU_LH=1, IR_Enable=0; PC incremented again.
T_2	Execute: Evaluate branch condition from flags (Z, C, N, O). If taken: MuxBSel=2'b10 (zero-extended IR[7:0]), ARF_RegSel=3'b110 (PC), ARF_FunSel=0 (Load). If not taken: assert T_Reset only. Either way, T_Reset=1.

The seven branch conditions are decoded as follows:

Table 5: Branch condition decoding.

Opcode	Mnemonic	Condition
0x00	BRA	Always taken
0x01	BNE	$Z = 0$
0x02	BEQ	$Z = 1$
0x03	BLT	$N \neq O$
0x04	BGT	$N = O$ and $Z = 0$
0x05	BLE	$N \neq O$ or $Z = 1$
0x06	BGE	$N = O$

2.5.2 Single-Operand ALU Instructions (Opcodes 0x07–0x0E)

These instructions read SREG1, apply a unary ALU operation, and write the result to DSTREG. They always finish at T_2 .

Table 6: Cycle trace for single-operand ALU instructions.

Cycle	Actions
T_0 – T_1	Standard two-cycle fetch (same as Table 1).
T_2	Route SREG1 to ALU input A via MuxC. Set ALU_FunSel to the corresponding operation (see Table 7). Assert ALU_WF=1 to update flags. Route ALU output to DSTREG through MuxA (if RF) or MuxB (if ARF). Assert T_Reset=1.

Table 7: Opcode to ALU function select mapping for single-operand instructions.

Opcode	Mnemonic	ALU_FunSel	Operation
0x07	INC	0x10	$A + 1$
0x08	DEC	0x11	$A - 1$
0x09	LSL	0x0B	Logical shift left
0x0A	LSR	0x0C	Logical shift right
0x0B	ASR	0x0D	Arithmetic shift right
0x0C	CSL	0x0E	Circular shift left
0x0D	CSR	0x0F	Circular shift right
0x0E	NOT	0x02	Bitwise NOT

2.5.3 Two-Operand ALU Instructions (Opcodes 0x0F–0x15)

Two-operand instructions need both ALU inputs occupied simultaneously. When SREG2 resides in the Register File this is straightforward—RF provides both `OutA` (for SREG1 via `MuxC`) and `OutB` (for SREG2) in the same cycle. However, when SREG2 is in the Address Register File, the control unit must first copy it into scratch register S1 and then perform the operation in a second execute cycle.

Table 8: Cycle trace for two-operand ALU instructions.

Cycle	Actions
T_0 – T_1	Standard fetch.
T_2	<i>Case A (SREG2 in RF):</i> Both operands available; execute ALU operation, write result, <code>T_Reset=1</code>. <i>Case B (SREG2 in ARF):</i> Copy SREG2 through <code>ARF_OutC</code> → <code>MuxC</code> → ALU (pass-through) → <code>MuxA</code> → scratch register S1 (<code>RF_ScrSel=4'b1110</code>).
T_3	<i>Case B only:</i> Read S1 on <code>OutB</code> , route SREG1 to ALU input A, perform operation, write DSTREG, <code>T_Reset=1</code> .

Table 9: ALU function mapping for two-operand instructions.

Opcode	Mnemonic	ALU_FunSel	Operation
0x0F	AND	0x07	A AND B
0x10	OR	0x08	A OR B
0x11	XOR	0x09	A XOR B
0x12	NAND	0x0A	A NAND B
0x13	ADD	0x04	A + B
0x14	ADC	0x05	A + B + Carry
0x15	SUB	0x06	A – B

2.5.4 Data Movement and Memory Instructions

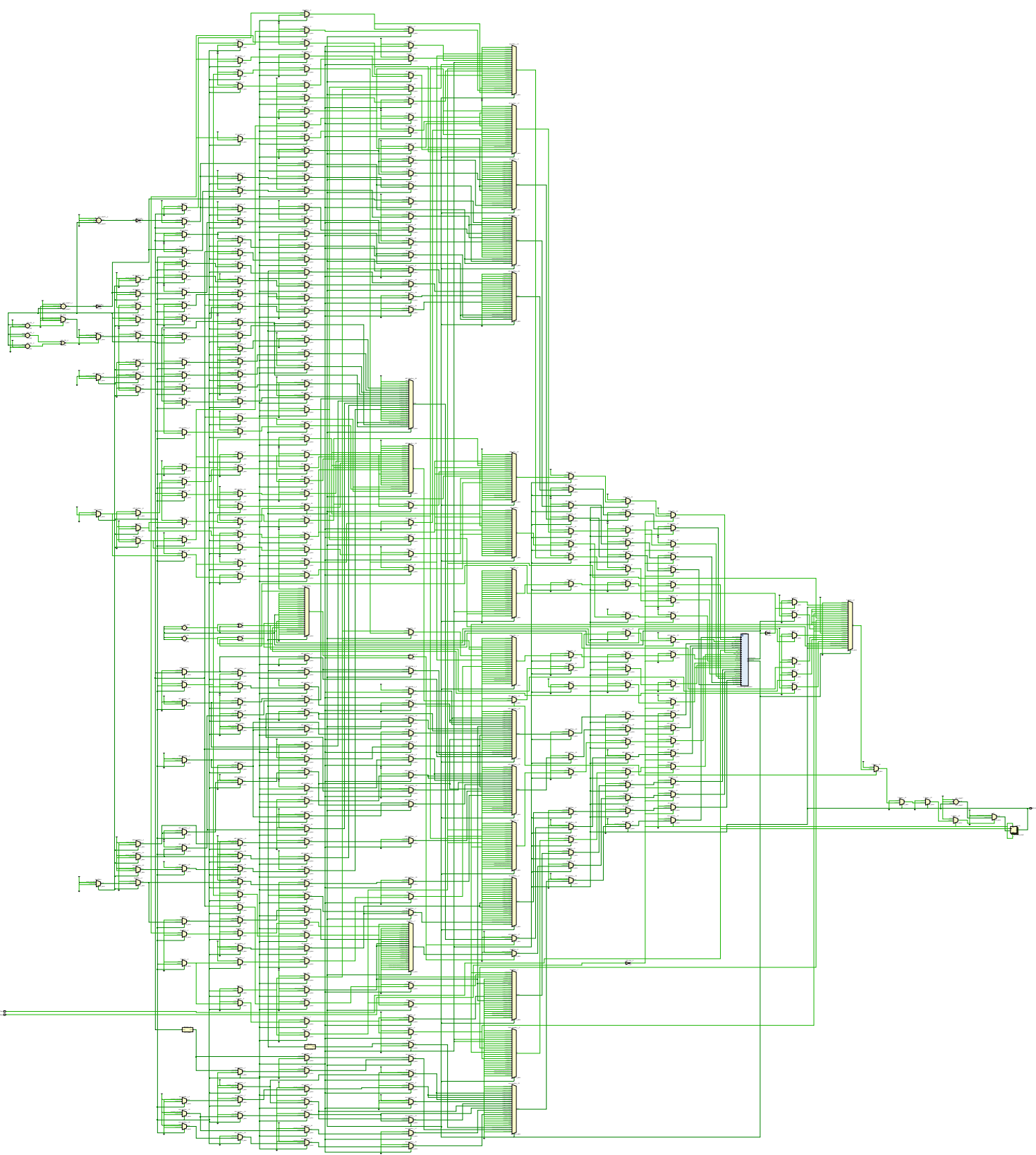
The remaining instructions involve register-to-register transfers, immediate loads, stack operations, and memory accesses. Their cycle counts vary from one to five execute cycles depending on the number of memory transactions required.

Table 10: Execute-phase cycle counts for data movement and memory instructions.

Opcode	Mnemonic	Exec. Cycles	Summary
0x16	MOV	1 (T_2)	SREG1 $\rightarrow$ ALU pass-through $\rightarrow$ DSTREG.
0x17	IMM	1 (T_2)	Zero-extended IR[7:0] via MuxA $\rightarrow$ Rx.
0x18	POP	4 (T_2 – T_5)	SP++; read LSB at M[SP], SP++; read MSB; DR $\rightarrow$ Rx.
0x19	PSH	4 (T_2 – T_5)	Write MSB to M[SP]; SP--; write LSB to M[SP]; SP--.
0x1A	CALL	5 (T_2 – T_6)	Push PC (MSB then LSB); PC $\leftarrow$ address.
0x1B	RET	4 (T_2 – T_5)	SP++; read LSB, SP++; read MSB; DR $\rightarrow$ PC.
0x1C	LDR	3 (T_2 – T_4)	Read M[AR] LSB, AR++; read MSB; DR $\rightarrow$ DSTREG.
0x1D	STR	3–4	Write SREG1 LSB/MSB to M[AR] (extra cycle if ARF source).
0x1E	LDA	4 (T_2 – T_5)	AR $\leftarrow$ address; read LSB, AR++; read MSB; DR $\rightarrow$ Rx.
0x1F	STA	4 (T_2 – T_5)	AR $\leftarrow$ address; write LSB; AR++; write MSB.
0x20	LDT	5 (T_2 – T_6)	S1 $\leftarrow$ offset; AR $\leftarrow$ AR+S1; read LSB, AR++; read MSB; DR $\rightarrow$ Rx.
0x21	STT	5 (T_2 – T_6)	S1 $\leftarrow$ offset; AR $\leftarrow$ AR+S1; write LSB; AR++; write MSB.

2.6 CPU System Schematic

The diagram shown below is that of all the data paths and control connections. Data multiplexers send data to/from the following units: Register File, Address Register File, ALU, instruction/data memory units, and Instruction Register.



2.7 Summary of Control Signal Generation

Table 11 provides a compact reference of every control signal, its width, and its role within the data-path.

Table 11: Control signals generated by the hardwired control unit.

Signal	Width	Purpose
RF_RegSel	4	Active-low enable for R1–R4.
RF_ScrSel	4	Active-low enable for S1–S4.
RF_FunSel	1	0 = Load, 1 = Clear.
RF_OutASel	3	Selects register for Output A.
RF_OutBSel	3	Selects register for Output B.
ARF_RegSel	3	Active-low enable for PC, AR, SP.
ARF_FunSel	1	0 = Load, 1 = Clear.
ARF_OutCSel	2	Selects ARF register for Output C.
ARF_OutDSel	2	Selects ARF register for Output D (address bus).
ALU_FunSel	5	Selects ALU operation (18 functions).
ALU_WF	1	Write-flag enable for Z, C, N, O.
MuxASel	2	RF input source: ALUOut / DR / IR[7:0] / OutC.
MuxBSel	2	ARF input source: ALUOut / DR / IR[7:0] / OutC.
MuxCSel	1	ALU input A source: RF_OutA / ARF_OutC.
IMU_CS	1	Instruction memory chip select (active-low).
IMU_LH	1	IR load half: 0 = low byte, 1 = high byte.
IR_Enable	1	IR write enable (active-low).
DMU_CS	1	Data memory chip select (active-low).
DMU_WR	1	Data memory write enable.
DMU_LH	1	DR load half: 0 = low byte, 1 = high byte.
DR_Enable	1	Data register enable (active-low).
DMU_DataSel	1	Selects ALUOut[7:0] or ALUOut[15:8] for memory write.
T_Reset	1	Resets timing counter to T_0 .

3 INSTRUCTION AND CLOCK CYCLE ANALYSIS

3.1 Complete Instruction Set with Cycle Counts

Table 12 provides the full list of instructions available to the CPU, together with the exact number of cycles required for their execution, counting from the beginning of the fetch cycle (T_0) up to the activation of **T.Reset**. The fetch cycle is guaranteed to take up at least two cycles (T_0 and T_1); all others are execution cycles.

Table 12: Complete instruction set with total clock cycle counts.

Opcode	Mnemonic	Type	Cycles	Description
0x00	BRA	Addr-ref	3	Unconditional branch
0x01	BNE	Addr-ref	3	Branch if $Z = 0$
0x02	BEQ	Addr-ref	3	Branch if $Z = 1$
0x03	BLT	Addr-ref	3	Branch if $N \neq O$
0x04	BGT	Addr-ref	3	Branch if $N = O$ and $Z = 0$
0x05	BLE	Addr-ref	3	Branch if $N \neq O$ or $Z = 1$
0x06	BGE	Addr-ref	3	Branch if $N = O$
0x07	INC	Reg-ref	3	$\text{DSTREG} \leftarrow \text{SREG1} + 1$
0x08	DEC	Reg-ref	3	$\text{DSTREG} \leftarrow \text{SREG1} - 1$
0x09	LSL	Reg-ref	3	Logical shift left of SREG1
0x0A	LSR	Reg-ref	3	Logical shift right of SREG1
0x0B	ASR	Reg-ref	3	Arithmetic shift right of SREG1
0x0C	CSL	Reg-ref	3	Circular shift left of SREG1
0x0D	CSR	Reg-ref	3	Circular shift right of SREG1
0x0E	NOT	Reg-ref	3	Bitwise NOT of SREG1
0x0F	AND	Reg-ref	3 / 4*	SREG1 AND SREG2
0x10	OR	Reg-ref	3 / 4*	SREG1 OR SREG2
0x11	XOR	Reg-ref	3 / 4*	SREG1 XOR SREG2

Continued on next page

Opcode	Mnemonic	Type	Cycles	Description
0x12	NAND	Reg-ref	3 / 4*	SREG1 NAND SREG2
0x13	ADD	Reg-ref	3 / 4*	SREG1 + SREG2
0x14	ADC	Reg-ref	3 / 4*	SREG1 + SREG2 + Carry
0x15	SUB	Reg-ref	3 / 4*	SREG1 − SREG2
0x16	MOV	Reg-ref	3	DSTREG ← SREG1
0x17	IMM	Addr-ref	3	Rx ← zero-extended VALUE
0x18	POP	Addr-ref	6	Pop 16-bit word from stack into Rx
0x19	PSH	Addr-ref	6	Push Rx onto stack (16- bit)
0x1A	CALL	Addr-ref	7	Push PC, jump to AD- DRESS
0x1B	RET	Addr-ref	6	Pop return address into PC
0x1C	LDR	Reg-ref	5	DSTREG ← M[AR] (16- bit)
0x1D	STR	Reg-ref	5 / 6†	M[AR] ← SREG1 (16-bit)
0x1E	LDA	Addr-ref	6	Rx ← M[ADDRESS] (16- bit)
0x1F	STA	Addr-ref	6	M[ADDRESS] ← Rx (16- bit)
0x20	LDT	Addr-ref	7	Rx ← M[AR + OFFSET]
0x21	STT	Addr-ref	7	M[AR + OFFSET] ← Rx

* Two-operand ALU instructions take 3 cycles when SREG2 is in the RF, or 4 cycles when SREG2 is in the ARF (extra cycle to copy through scratch register S1).

† STR takes 5 cycles when SREG1 is in the RF, or 6 cycles when SREG1 is in the ARF (extra copy to S1).

3.2 Reasoning Behind Cycle Counts

The cycle count for each instruction is driven by a few hardware constraints:

1. **Instruction fetch overhead (2 cycles).** Every instruction pays a fixed two-cycle cost because the 8-bit-wide instruction memory requires separate reads for the LSB and MSB halves of the 16-bit instruction word.

2. **Single-cycle ALU propagation.** The ALU is purely combinational: once its inputs are stable on MuxC and OutB, the result appears at ALUOut within the same cycle. Single-operand and register-only two-operand instructions therefore complete in one execute cycle.
3. **ARF-to-RF copy penalty.** The ALU's B input comes exclusively from the Register File's OutB port. When SREG2 resides in the ARF, the control unit must spend one cycle copying the value into scratch register S1 so that it can be read from OutB in the following cycle.
4. **Byte-wide data memory.** The data memory is also 8 bits wide. Reading or writing a 16-bit value requires two separate memory transactions (one for the low byte and one for the high byte), each consuming one cycle. Instructions that access data memory therefore need at least two execute cycles for the memory operation alone.
5. **Stack pointer adjustment.** Stack operations (POP, PSH, CALL, RET) must update SP between each byte-level memory access. Each SP increment or decrement occupies an additional cycle because the ALU result must be written back to the ARF before the next memory address can be driven.
6. **Address register setup.** Instructions such as LDA, STA, LDT, and STT need to load or compute the effective address into AR before starting the memory transaction. LDA and STA spend one cycle loading the immediate address into AR. LDT and STT additionally copy the offset into S1 and perform an addition with the current AR value, consuming two cycles for address computation.

4 SIMPLE PROGRAM ANALYSIS

4.1 Program Description

This program written in assembly language, depicted in Fig. 1, is executed from memory location 0x8C. First, the program loads the iteration count to register R1 using `IMM R1, 0x06`, and the base address of the data array to register R3 using `IMM R3, 0xB0`. Subsequently, register R3 is copied to AR using `MOV AR, R3` to keep track of the current position in the array. Finally, the first element of the array is loaded to register R2 through `LDR R2`.

The commands `INC AR, AR` and `DEC R1, R1` move the pointer and decrease the iteration count, after which execution goes into the main loop. The next element of the array is read into R4 (`LDR R4`) and `SUB R3, R4, R2` calculates the difference between the new element and the maximum element. If the latter does not hold (`BLE SKIP`), the program jumps to `SKIP`; otherwise, `MOV R2, R4` is performed. Both options join at the point `SKIP`, where AR and R1 are increased and decreased, respectively, and the `BNE LABEL` jumps back as long as the iteration counter holds non-zero values. Finally, outside the loop, `STR R2` is executed, and the maximum number is stored in the location specified by `AR = 0xBC`.

In short, the program scans six 16-bit values stored at addresses 0xB0–0xBB, determines the maximum, and stores the result at address 0xBC.

```
BRA    0x8C      # This instruction is written to the memory address 0x00,
                # The first instruction must be written to address 0x8C

IMM    R1, 0x06   # R1 is used for iteration number
IMM    R3, 0xB0

MOV    AR, R3     # AR is used to track data address: starts from 0xB0
LDR    R2         # R2 ← M[AR]
INC    AR, AR     # AR ← AR + 1 (Next Data)
DEC    R1, R1     # R1 ← R1 - 1 (Decrement Iteration Counter)
LABEL: LDR    R4   # R4 ← M[AR]
SUB    R3, R4, R2
BLE    SKIP      # SKIP if N!=0 | Z==1
MOV    R2, R4
SKIP:  INC    AR, AR # AR ← AR + 1 (Next Data)
DEC    R1, R1     # R1 ← R1 - 1 (Decrement Iteration Counter)
BNE    LABEL     # Go back to LABEL if Z=0 (Iteration Counter > 0)
STR    R2        # M[AR] ← R2 (Store result at 0xBC)
```

Figure 1: Assembly listing of the simple “find maximum” program.

4.2 Total Clock Cycle Calculation

The cycle count is computed by tracing through the program instruction by instruction. The data at addresses 0xB0–0xBB in RAM (little-endian 16-bit words) consists of the values 0x00B6, 0x00B3, 0x00BB, 0x00B0, 0x00B8, 0x00B5. The maximum is 0x00BB (the third element).

4.2.1 Initialization Phase (before loop)

Table 13: Cycle breakdown for the initialization phase.

Address	Instruction	Cycles
0x8C	BRA 0x8C (jump to start)	3
0x8C	IMM R1, 0x06	3
0x8E	IMM R3, 0xB0	3
0x90	MOV AR, R3	3
0x92	LDR R2	5
0x94	INC AR, AR	3
0x96	DEC R1, R1	3
Initialization subtotal		23

4.2.2 Loop Body

Each iteration of the loop executes the following sequence. The branch outcome for BLE SKIP depends on whether the new element exceeds the current maximum.

Table 14: Per-iteration cycle breakdown of the main loop.

Label	Instruction	Cycles	Notes
LABEL:	LDR R4	5	Load next element
	SUB R3, R4, R2	3	Compare (RF×RF)
	BLE SKIP	3	Conditional skip
	MOV R2, R4	3	Only if R4 > R2
SKIP:	INC AR, AR	3	Advance pointer
	DEC R1, R1	3	Decrement counter
	BNE LABEL	3	Loop back if R1 ≠ 0
Iteration with update (BLE not taken)		23	
Iteration without update (BLE taken)		20	

The six data values are 0x00B6, 0x00B3, 0x00BB, 0x00B0, 0x00B8, 0x00B5. R2 starts with the first value (0x00B6) loaded during initialization, so the loop runs 5 times (R1 counts down from 5 to 0):

- **Iteration 1** (R1=5): Compare 0x00B3 vs. 0x00B6 → not greater → 20 cycles.
- **Iteration 2** (R1=4): Compare 0x00BB vs. 0x00B6 → greater, update R2 → 23 cycles.
- **Iteration 3** (R1=3): Compare 0x00B0 vs. 0x00BB → not greater → 20 cycles.
- **Iteration 4** (R1=2): Compare 0x00B8 vs. 0x00BB → not greater → 20 cycles.
- **Iteration 5** (R1=1): Compare 0x00B5 vs. 0x00BB → not greater → 20 cycles. BNE falls through (R1=0 after DEC, so Z=1).

The last iteration's BNE is not taken (Z=1), but it still costs 3 cycles; however, it does *not* branch back, so the loop exits after that cycle.

4.2.3 Finalization

Table 15: Post-loop instruction.

Instruction	Description	Cycles
STR R2	Store max at M[AR] (0xBC)	5

4.2.4 Grand Total

$$\underbrace{23}_{\text{Init}} + \underbrace{20 + 23 + 20 + 20 + 20}_{5 \text{ loop iterations}} + \underbrace{5}_{\text{STR}} = \mathbf{131} \text{ clock cycles}$$

The program completes in approximately **131 clock cycles**, writing the maximum value 0x00BB to memory address 0xBC.

5 SIMULATION RESULTS

Two different testbenches for testing the CPU have been created. These include per instruction testbench for CPU (`CPUSystemSimulation`) and factorial calculation of CPU (`CPUSystemSimulationFactorial`). Both tests have been executed on the simulator **Xilinx Vivado XSIM**.

5.1 Per-Instruction Simulation

The per-instruction testbench exercises critical instructions in isolation by pre-loading register values, setting the IR to a specific opcode, and advancing the T counter until the instruction completes. After each test the expected register and memory values are compared against actual outputs.

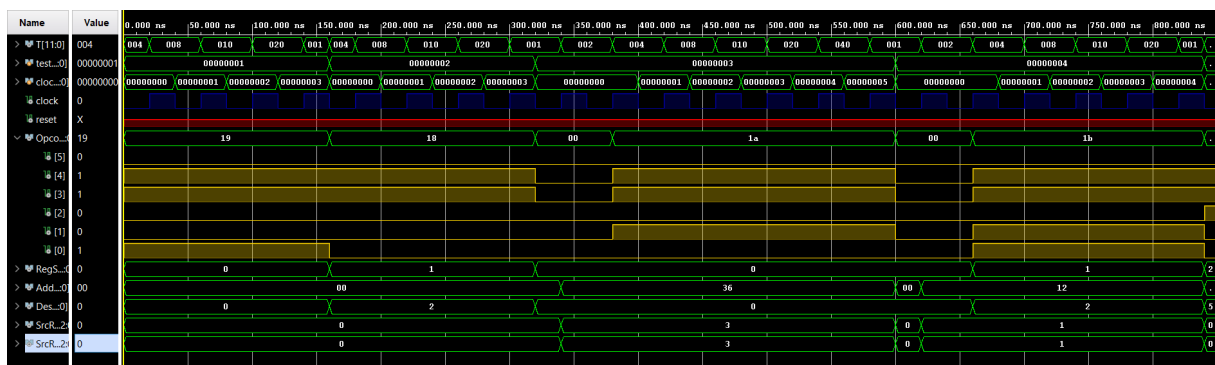


Figure 2: Waveform output from the per-instruction CPU system simulation showing sequential test execution.

Key observations from the waveform:

- **Push-pop stack operation (PSH/POP):** During push operation (PSH), the SP register is decremented while data is written in MSB and LSB order to two successive memory addresses. For pop operation, the SP register is incremented while data is read from memory. The data read from memory matches the original data pushed to the stack.
- **CALL/RET pair:** In CALL, the program counter's current value is pushed on the stack with a total of two byte writes, then the target address is loaded into PC. RET pops two bytes from the stack and loads them into PC. The simulation proves that following a CALL/RET sequence, the program counter will be reset to the next instruction after the initial CALL.
- **LDR/STR memory accesses:** In an LDR instruction, two consecutive bytes are fetched from the data memory, beginning from the address stored in the AR register, combined as a 16-bit word using the Data Register, and stored in the target register. The operation is reversed for STR. One finds that after fetching the low byte, AR increments by one such that the high byte fetches the correct adjacent memory location.

- **LDA/STA with immediate addressing mode:** These instructions initially load the address byte to the register AR, after which they perform the regular two-cycle sequence for loading/storing data from/to memory. The additional cycle needed for address loading can be seen in the waveform as one more T-state prior to the actual memory operation.
- **LDT / STT with Offset:** The offset is placed in scratch register S1, and the addition operation takes place in the ALU, where it is sent to AR for further use. Waveform analysis shows that both the addition and byte-level memory transactions have been successfully performed.

```

-----
CPUSystemSimulation Simulation Started
[PASS] Test No: 1, Component: MEM[SP, FE], Actual Value: 0x0012, Expected Value: 0x0012
[PASS] Test No: 1, Component: MEM[SP, FF], Actual Value: 0x0023, Expected Value: 0x0023
[PASS] Test No: 1, Component: SP, Actual Value: 0x00fd, Expected Value: 0x00fd
[PASS] Test No: 2, Component: MEM[SP, FE], Actual Value: 0x0012, Expected Value: 0x0012
[PASS] Test No: 2, Component: MEM[SP, FF], Actual Value: 0x0023, Expected Value: 0x0023
[PASS] Test No: 2, Component: R1, Actual Value: 0x2312, Expected Value: 0x2312
[PASS] Test No: 2, Component: SP, Actual Value: 0x00ff, Expected Value: 0x00ff
[PASS] Test No: 3, Component: PC, Actual Value: 0x0036, Expected Value: 0x0036
[PASS] Test No: 3, Component: MEM[SP, FC], Actual Value: 0x0070, Expected Value: 0x0070
[PASS] Test No: 3, Component: MEM[SP, FD], Actual Value: 0x0000, Expected Value: 0x0000
[PASS] Test No: 3, Component: SP, Actual Value: 0x00fb, Expected Value: 0x00fb
[PASS] Test No: 4, Component: PC, Actual Value: 0x0070, Expected Value: 0x0070
[PASS] Test No: 4, Component: SP, Actual Value: 0x00fd, Expected Value: 0x00fd
[PASS] Test No: 5, Component: AR, Actual Value: 0x0007, Expected Value: 0x0007
[PASS] Test No: 5, Component: R2, Actual Value: 0x0703, Expected Value: 0x0703
[PASS] Test No: 6, Component: AR, Actual Value: 0x0015, Expected Value: 0x0015
[PASS] Test No: 6, Component: MEM[AR, 14], Actual Value: 0x0022, Expected Value: 0x0022
[PASS] Test No: 6, Component: MEM[AR, 15], Actual Value: 0x0011, Expected Value: 0x0011
[PASS] Test No: 7, Component: AR, Actual Value: 0x0021, Expected Value: 0x0021
[PASS] Test No: 7, Component: R4, Actual Value: 0x1405, Expected Value: 0x1405
[PASS] Test No: 8, Component: AR, Actual Value: 0x00a9, Expected Value: 0x00a9
[PASS] Test No: 8, Component: MEM[AR, A8], Actual Value: 0x0089, Expected Value: 0x0089
[PASS] Test No: 8, Component: MEM[AR, A9], Actual Value: 0x0067, Expected Value: 0x0067
[PASS] Test No: 9, Component: AR, Actual Value: 0x0017, Expected Value: 0x0017
[PASS] Test No: 9, Component: R2, Actual Value: 0x0703, Expected Value: 0x0703
[PASS] Test No: 10, Component: AR, Actual Value: 0x00c2, Expected Value: 0x00c2
[PASS] Test No: 10, Component: MEM[AR, C1], Actual Value: 0x0011, Expected Value: 0x0011
[PASS] Test No: 10, Component: MEM[AR, C2], Actual Value: 0x0000, Expected Value: 0x0000
CPUSystemSimulation Simulation Finished
0 Test Failed
28 Test Passed
-----

```

Figure 3: All per-instruction simulation tests passed—console output confirms zero failures.

5.2 Factorial Program Simulation

The factorial testbench runs a complete recursive program that computes $4! = 24$ (0x18 in hexadecimal). This program uses CALL/RET for recursion, PSH/POP to save and restore registers across recursive calls, and a multiply subroutine implemented via repeated addition.

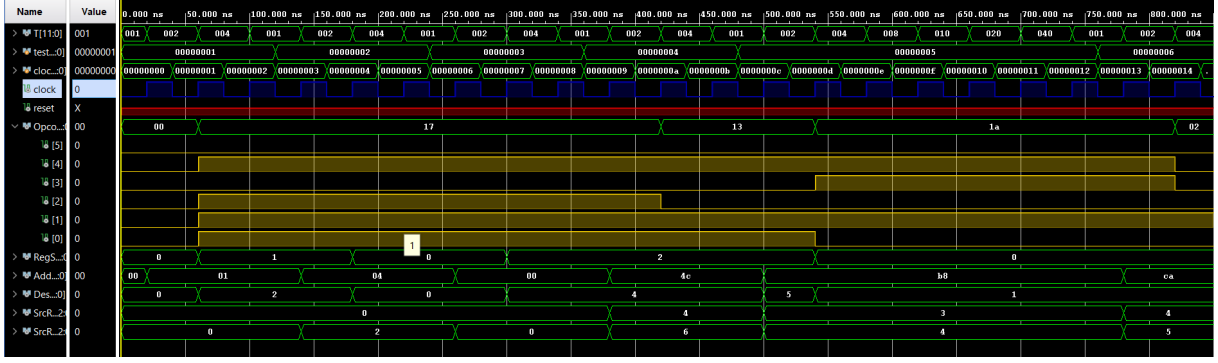


Figure 4: Waveform from the factorial (4!) program simulation.

The simulation confirms the following end-state values:

- R2 = 0x0018 (decimal 24, i.e. 4!), stored at memory address 0x54.
- The stack pointer returns to 0x00FF after all recursive frames have been unwound, demonstrating that every PSH is matched by a corresponding POP or RET.
- The stack contents at addresses 0xF2–0xFF contain the saved return addresses from each level of recursion, matching the expected call sequence.

```

-----
CPUSystemSimulationFactorial Simulation Started
[PASS] Test No: 70, Component: IROut, Actual Value: 0x00d8, Expected Value: 0x00d8
[PASS] Test No: 70, Component: PC, Actual Value: 0x00d8, Expected Value: 0x00d8
[PASS] Test No: 70, Component: SP, Actual Value: 0x00ff, Expected Value: 0x00ff
[PASS] Test No: 70, Component: AR, Actual Value: 0x0055, Expected Value: 0x0055
[PASS] Test No: 70, Component: MEM[ADDRESS], Actual Value: 0x0018, Expected Value: 0x0018
[PASS] Test No: 70, Component: MEM[ADDRESS], Actual Value: 0x0000, Expected Value: 0x0000
[PASS] Test No: 70, Component: R1, Actual Value: 0x0001, Expected Value: 0x0001
[PASS] Test No: 70, Component: R2, Actual Value: 0x0018, Expected Value: 0x0018
[PASS] Test No: 70, Component: R3, Actual Value: 0x0006, Expected Value: 0x0006
[PASS] Test No: 70, Component: R4, Actual Value: 0x0000, Expected Value: 0x0000
[PASS] Test No: 70, Component: MEM[SP], Actual Value: 0x00c6, Expected Value: 0x00c6
[PASS] Test No: 70, Component: MEM[SP], Actual Value: 0x0000, Expected Value: 0x0000
[PASS] Test No: 70, Component: MEM[SP], Actual Value: 0x00ca, Expected Value: 0x00ca
[PASS] Test No: 70, Component: MEM[SP], Actual Value: 0x0000, Expected Value: 0x0000
[PASS] Test No: 70, Component: MEM[SP], Actual Value: 0x00c6, Expected Value: 0x00c6
[PASS] Test No: 70, Component: MEM[SP], Actual Value: 0x0000, Expected Value: 0x0000
[PASS] Test No: 70, Component: MEM[SP], Actual Value: 0x00ca, Expected Value: 0x00ca
[PASS] Test No: 70, Component: MEM[SP], Actual Value: 0x0000, Expected Value: 0x0000
[PASS] Test No: 70, Component: MEM[SP], Actual Value: 0x00c6, Expected Value: 0x00c6
[PASS] Test No: 70, Component: MEM[SP], Actual Value: 0x0000, Expected Value: 0x0000
[PASS] Test No: 70, Component: MEM[SP], Actual Value: 0x00ca, Expected Value: 0x00ca
[PASS] Test No: 70, Component: MEM[SP], Actual Value: 0x0000, Expected Value: 0x0000
[PASS] Test No: 70, Component: MEM[SP], Actual Value: 0x00b4, Expected Value: 0x00b4
[PASS] Test No: 70, Component: MEM[SP], Actual Value: 0x0000, Expected Value: 0x0000
CPUSystemSimulationFactorial Simulation Finished
0 Test Failed
24 Test Passed

```

Figure 5: Factorial simulation test results—all checks passed.

These results demonstrate that the hardwired control unit correctly orchestrates multi-cycle instructions, manages the stack across nested subroutine calls, and handles flag-dependent branching without error.

6 DISCUSSION

This decision to use a hardwired control unit rather than microprogramming was made mainly because of performance and simplicity reasons, due to the reduced number of instructions in the instruction set. The absence of any overhead associated with fetching microinstructions and having a shorter critical path inside the control unit as compared to using a control memory make this choice clear.

One such compromise that occurred for instructions operating on two operands in ALU is in the case where the operand SREG2 comes from the Address Register File. As the operand at the input of B of ALU can only be supplied by the Output port of Register File (OutB), it is necessary to have one additional cycle in order to move the content of ARF into the scratch register S1. This may be avoided by using a larger MUX for B operand of ALU.

The 8-bit wide memory interface acts as the biggest limitation to performance for instructions that need to access data memory. All 16-bit read/write operations need two memory accesses each, along with any pointer adjustments, thereby doubling the number of clock cycles needed for instructions such as POP, PSH, CALL, and RET. However, a 16-bit memory interface will cut down the number of clock cycles by half.

However, this means that the maximum length of an instruction is restricted to 12 clock cycles since only one of the bits in the one-hot encoded form must be checked per T-state. The maximum number of cycles for any instruction currently available under our ISA is seven.

7 CONCLUSION

All the subsystems that were designed for Project 1 and Project 2 were integrated into this project using a hardwired control unit that could execute 34 different instructions. This has been tested using specific tests per instruction as well as a full recursion factorial test program, and everything went perfectly fine.

Generating control signals for each instruction and timing point took up the bulk of the design phase. Minor errors, like failing to enable `T_Reset` after completing an instruction or setting `DMU_DataSel` incorrectly for the high byte, would lead to errors in larger instruction sets. Thorough testbenching was indispensable in identifying such problems.

The first lesson learned from the experience was the significance of the `set_defaults` process. If the `set_defaults` function is not used, there might arise cases when a particular signal that is supposed to be off in a few T-states can take on the value from another branch of the combinational circuit.

Overall, the project provided a hands-on understanding of how a processor's control unit translates instruction opcodes into precise sequences of data-path operations across multiple clock cycles.

8 TASK DISTRIBUTION

Table 16: Distribution of tasks among group members.

Member	Responsibilities
Celal Gündoğdu(150230007)	State machine implementation and realization of the control unit (<code>CPUSystem.v</code>) such as T_0/T_1 fetching, one-hot timing counter, and <code>set_defaults</code> . Debugging and verification of branching instructions decoding (BRA to BGE) and control flow conditional on flags. Stack manipulation instructions PSH, POP, CALL, and RET implementation. Sections from the report about the control unit design and simple program analysis.
İsmail Efe Çelik(150230008)	ALU Integration with Data Path Multiplexers, guaranteeing proper routing for one and two operand instructions as well as ARF to Scratch Register work-around. Optimization of clock cycles in Memory Instructions such as LDR, STR, LDA, STA, LDT, STT. Creation and implementation of Simulations Test Benches (<code>CPUSystemSimulation.v</code> and <code>CPUSystemSimulationFactorial.v</code>). Waveform analysis of Simulation outputs. Simulation Results and Instruction Cycle Analysis Sections of the report.

REFERENCES

- [1] M. M. Mano and C. R. Kime, *Logic and Computer Design Fundamentals*, 5th ed. Pearson, 2015.
- [2] D. A. Patterson and J. L. Hennessy, *Computer Organization and Design: The Hardware/Software Interface*, 5th ed. Morgan Kaufmann, 2014.
- [3] S. Harris and D. Harris, *Digital Design and Computer Architecture*, 2nd ed. Morgan Kaufmann, 2012.
- [4] F. Vahid, *Digital Design with RTL Design, VHDL, and Verilog*, 2nd ed. John Wiley & Sons, 2010.
- [5] IEEE, “IEEE Standard for Verilog Hardware Description Language,” *IEEE Std 1364-2005*, 2006.
- [6] Xilinx, “Vivado Design Suite User Guide: Logic Simulation,” UG900 (v2023.2), 2023. [Online]. Available: <https://docs.amd.com/r/en-US/ug900-vivado-logic-simulation>